"Para quem só sabe usar martelo, todo problema é um prego" (Abraham Maslow).

Orientação a Objetos Parte 2

Paulo Lisboa de Almeida

GenAI

UFPR

Exceções

Exceções

Na orientação a objetos temos o conceito de **exceção**.

Indicar que algo deu errado.

- Mecanismo padrão para tratamento de erros.
- Criar programas tolerantes a falhas.
- Detectar uma falha específica e se recuperar.
- Facilitar encontrar a fonte de um problema.

Exceções

Uma **exceção** deve ser **lançada** quando um problema acontece.

Algo inesperado, ou que não deveria acontecer no fluxo normal do programa.

CPF

Vamos melhorar a O.O. lançando uma exceção caso um cpf inválido seja passado.

Para lançar uma exceção, utilize a palavra-chave `raise`.

raise

```
class Pessoa:
  def __init__(self, nome=None, cpf=-1):
      self.__nome = nome
      if cpf != -1 and self.__validar_cpf(cpf):
          self.__cpf = cpf
      else:
          raise ValueError("CPF Inválido")

  @property
  def cpf(self):
      return self.__cpf

  @cpf.setter
  def cpf(self, cpf):
      if self.__validar_cpf(cpf):
          self.__cpf = cpf
      else:
          raise ValueError("CPF Inválido")
```

raise

Quando um `raise` é executado.

O objeto do tipo da exceção é imediatamente lançado para que alguém o pegue.

A função é abortada imediatamente.

- Não termina sua execução.
- Não retorna resultados.

raise

```
class Pessoa:
  def __init__(self, nome=None, cpf=-1):
      self.__nome = nome
      if cpf != -1 and self.__validar_cpf(cpf):
          self.__cpf = cpf
      else:
          raise ValueError("CPF Inválido")

  @property
  def cpf(self):
      return self.__cpf

  @cpf.setter
  def cpf(self, cpf):
      if self.__validar_cpf(cpf):
          self.__cpf = cpf
      else:
          raise ValueError("CPF Inválido")
```

Uma exceção do tipo ValueError será lançada.
Lance exceções dos tipos corretos!
Veja a lista de exceções padrão: docs.python.org/3/library/exceptions.html

main.py

```python
from universidade.pessoa import Pessoa

p1 = Pessoa("Maria", 11111111111)

p1.cpf = 123

print(p1.nome)
print(p1.cpf)
```

try-except

Execute o trecho do código que pode lançar exceções em um bloco `try-except`.

O trecho deve ficar dentro de um bloco `try`.

Após o try, adicionamos um bloco `except` (tipo nomeExceção).

O catch "pega" a exceção lançada.

main.py

```python
from universidade.pessoa import Pessoa

try:
  p1 = Pessoa("Maria", 11111111111)
  p1.cpf = 123

  print(p1.nome)
  print(p1.cpf)
except Exception as e:
  #Faz algo útil aqui, como informar o usuário ou logar o erro
  print("O CPF não é válido!")
  print(e)
```

main.py

```python
from universidade.pessoa import Pessoa

try:
  p1 = Pessoa("Maria", 11111111111)
  p1.cpf = 123

  print(p1.nome)
  print(p1.cpf)
except Exception as e:
  #Faz algo útil aqui, como informar o usuário ou logar o erro
  print("O CPF não é válido!")
  print(e)
```

Pegue (catch) qualquer exceção genérica.

main.py

```python
from universidade.pessoa import Pessoa

try:
  p1 = Pessoa("Maria", 11111111111)
  p1.cpf = 123

  print(p1.nome)
  print(p1.cpf)
except ValueError as e:
  #Faz algo útil aqui, como informar o usuário ou logar o erro
  print("O CPF não é válido!")
  print(e)
```

Pegue apenas exceções do tipo ValueError.

try-except

1. O `try` tenta executar o seu bloco.
2. Se nenhuma exceção for lançada, o `try` é executado completamente.
    a. O `except` não é executado.
3. Se uma exceção é **lançada**.
    a. As instruções do `try` param de ser executadas imediatamente.
    b. Se o `except` espera essa exceção.
        i. O `except` relacionado a exceção a **pega**, e é executado.

Exceções Customizadas

Você pode criar exceções customizadas.

Geralmente é uma boa ideia.

Por exemplo, criar uma exceção chamada CPFInvalidoExcetiption.

Toda exceção customizada deve derivar de **Exception**.

Precisa de herança. Veremos adiante.

Pesquise

É possível (e uma boa prática) encadear múltiplos except depois de um try.

Cada except pode pegar uma exceção diferente, e fazer o tratamento de forma adequada.

Verifique na literatura e documentação como fazer.

Herança

Herança

Com a herança podemos criar classes que herdam (absorvem) as capacidades de classes já existentes.

Salvamos tempo.

- Não precisamos recriar tudo do zero.

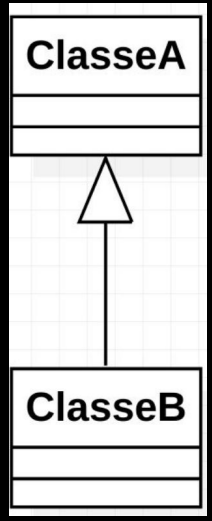
Criamos software de qualidade.

- Utilizamos classes já existentes e testadas como base.

O desenvolvimento se torna mais simples.

- Provemos abstração.

Herança



Uma classe B herda os membros da classe A.

A classe A é chamada de **classe base** ou **classe pai**.

A classe B é chamada de **classe derivada** ou **classe filha**.

A classe B deriva de A.

Uma classe derivada representa uma especialização da classe base.

Formamos hierarquias de classes.

Exemplo

Desejamos fazer a distinção entre alunos e professores.

Alunos possuem:

- Número de matrícula.
- Curso onde estão matriculados.

Professores possuem:

- Carga Horária Semanal.
- Salário.

Mas tanto professores quanto alunos são **tipos de** Pessoa.

Possuem nome, cpf, ...

Exemplo

Desejamos fazer a distinção entre alunos e professores.

Alunos possuem:

- Número de matrícula.
- Curso onde estão matriculados.

Professores possuem:

- Carga Horária Semanal.
- Salário.

Mas tanto professores quanto alunos são **tipos de** Pessoa.

Possuem nome, cpf, ...

Copiar e colar a classe Pessoa para criar duas especializações seria uma **péssima ideia**.

Professor.py

```
from .pessoa import Pessoa

class Professor(Pessoa):
  def __init__(self, nome=None, cpf=-1):
      super().__init__(nome, cpf)
```

Professor.py

```
from .pessoa import Pessoa

class Professor(Pessoa):
  def __init__(self, nome=None, cpf=-1):
      super().__init__(nome, cpf)
```

A classe Professor herda de Pessoa.

Professor é um tipo de Pessoa.

Professor.py

```python
from .pessoa import Pessoa

class Professor(Pessoa):
  def __init__(self, nome=None, cpf=-1):
      super().__init__(nome, cpf)
```

No Python use `super()` para invocar algo da classe Pai.
Aqui, estamos invocando o inicializador passando o nome e cpf.

main.py

Como Professor herda de Pessoa, ele consegue fazer tudo o que uma Pessoa faz.

Note que ele possui um nome, mesmo isso não sendo definido diretamente na classe Professor.

```
from .pessoa import Pessoa

class Professor(Pessoa):
  def __init__(self, nome=None, cpf=-1):
      super().__init__(nome, cpf)
```

```
from universidade.professor import Professor

professor1 = Professor("Paulo", 11111111111)

print(professor1.nome)
```

Tipo de

Professor é um **tipo de** Pessoa.

Possui tudo o que uma pessoa possui (nome, cpf, ...).

Tipo de

Professor é um **tipo de** Pessoa.

Possui tudo o que uma pessoa possui (nome, cpf, ...).

Relações tipo de **não são reflexivas**!

Todo Professor é uma Pessoa (é um tipo de pessoa, ou uma especialização de pessoa).

Nem toda Pessoa é um tipo de Professor (nem toda pessoa é professor).

Aluno.py

```python
from .pessoa import Pessoa

class Aluno(Pessoa):
  def __init__(self, nome=None, cpf=-1):
      super().__init__(nome, cpf)
```

Protected

Professor.py

```
class Professor(Pessoa):
  def __init__(self, nome, cpf, valor_hora = 60, carga_horaria = 40):
     super().__init__(nome, cpf)
     self.__valor_hora = valor_hora
     self.__carga_horaria = carga_horaria

  @property
  def valor_hora(self):
      return self.__valor_hora

  @valor_hora.setter
  def valor_hora(self, valor_hora):
      self.__valor_hora = valor_hora

  @property
  def carga_horaria(self):
      return self.__carga_horaria

  @carga_horaria.setter
  def carga_horaria(self, carga_horaria):
      self.__carga_horaria = carga_horaria

  @property
  def salario(self):
      #Carga semanal * custo * 4,5 semanas no mês
      return int(self.__valor_hora * self.__carga_horaria * 4.5)
```

Professor.py

```
class Professor(Pessoa):
  def __init__(self, nome, cpf, valor_hora = 60, carga_horaria = 40):
      super().__init__(nome, cpf)
      self.__valor_hora = valor_hora
      self.__carga_horaria = carga_horaria

  @property
  def valor_hora(self):
      return self.__valor_hora

  @valor_hora.setter
  def valor_hora(self, valor_hora):
      self.__valor_hora = valor_hora

  @property
  def carga_horaria(self):
      return self.__carga_horaria

  @carga_horaria.setter
  def carga_horaria(self, carga_horaria):
      self.__carga_horaria = carga_horaria

  @property
  def salario(self):
      #Carga semanal * custo * 4,5 semanas no mês
      return int(self.__valor_hora * self.__carga_horaria * 4.5)
```

Estamos considerando o valor em reais, sem considerar centavos (números inteiros).

Professor.py

```
class Professor(Pessoa):
  def __init__(self, nome, cpf, valor_hora = 60, carga_horaria = 40):
      super().__init__(nome, cpf)
      self.__valor_hora = valor_hora
      self.__carga_horaria = carga_horaria

  @property
  def valor_hora(self):
      return self.__valor_hora

  @valor_hora.setter
  def valor_hora(self, valor_hora):
      self.__valor_hora = valor_hora

  @property
  def carga_horaria(self):
      return self.__carga_horaria

  @carga_horaria.setter
  def carga_horaria(self, carga_horaria):
      self.__carga_horaria = carga_horaria

  @property
  def salario(self):
      #Carga semanal * custo * 4,5 semanas no mês
      return int(self.__valor_hora * self.__carga_horaria * 4.5)
```

É uma má ideia representar valores monetários e afins com números “depois da vírgula” devido a erros numéricos da máquina causados pela representação em ponto flutuante. Veja em:

prlalmeida.com.br/2021/10/29/digital-circuits

prlalmeida.com.br/2022/08/29/computer-architecture

Professor.py

```
class Professor(Pessoa):
  def __init__(self, nome, cpf, valor_hora = 60, carga_horaria = 40):
     super().__init__(nome, cpf)
     self.__valor_hora = valor_hora
     self.__carga_horaria = carga_horaria

  @property
  def valor_hora(self):
      return self.__valor_hora

  @valor_hora.setter
  def valor_hora(self, valor_hora):
      self.__valor_hora = valor_hora

  @property
  def carga_horaria(self):
      return self.__carga_horaria

  @carga_horaria.setter
  def carga_horaria(self, carga_horaria):
      self.__carga_horaria = carga_horaria

  @property
  def salario(self):
      #Carga semanal * custo * 4,5 semanas no mês
      return int(self.__valor_hora * self.__carga_horaria * 4.5)
```

Mais uma boa ideia. Não existe o atributo salário. Mas podemos fazer um get que simula esse atributo, calculando-o automaticamente.

main.py

```python
from universidade.professor import Professor
from universidade.pessoa import Pessoa

try:
 p1 = Pessoa("Maria", 11111111111)
 t1 = Professor("João", 22222222222)

 print(p1.nome)
 print(p1.cpf)

 print(t1.nome)
 print(t1.cpf)
 print(t1.salario)
except ValueError as e:
 print(e)
```

main.py

Os atributos de Pessoa são **privados**.

Nem mesmo suas classes derivadas (Professor e Aluno) devem acessar diretamente os atributos.

```
class Pessoa:
  def __init__(self, nome=None, cpf=-1):
      self.__nome = nome
      if cpf != -1 and self.__validar_cpf(cpf):
          self.__cpf = cpf
      else:
          raise ValueError("CPF Inválido")
```

Protected

Podemos deixar os atributos **protegidos** (**protected**).

Um atributo protegido é similar a um privado.

Mas informamos que as classes filhas (derivadas) que elas podem acessar os atributos diretamente.

```
class Pessoa:
  def __init__(self, nome=None, cpf=-1):
      self._nome = nome
      if cpf != -1 and self.__validar_cpf(cpf):
          self._cpf = cpf
      else:
          raise ValueError("CPF Inválido")
```

issubclass

Verificando a hierarquia

Diferente de linguagens tipadas (C, C++, Java, ...), no Python precisamos verificar manualmente determinado parâmetro pertence a uma hierarquia.

Utilizamos as funções:

- `isinstance()`: verifica se a classe é de determinado tipo.
- `issubclass()`: verifica se a classe deriva de determinado tipo.

issubclass

```python
from .conteudo_ministrado import ConteudoMinistrado
from .professor import Professor

class Disciplina:
    def __init__(self, nome, carga_horaria = 60, professor=None):
        self.__nome = nome
        self.carga_horaria = carga_horaria
        self.professor = professor
        self.__conteudos_ministrados = []

    @property
    def professor(self):
        return self.__professor

    @professor.setter
    def professor(self, professor):
        if professor is not None and not issubclass(type(professor), Professor):
            raise TypeError("parâmetro professor deve derivar da classe Professor")
        self.__professor = professor

#...
```

issubclass

```python
from .conteudo_ministrado import ConteudoMinistrado
from .professor import Professor

class Disciplina:
    def __init__(self, nome, carga_horaria = 60, professor=None):
        self.__nome = nome
        self.carga_horaria = carga_horaria
        self.professor = professor
        self.__conteudos_ministrados = []

    @property
    def professor(self):
        return self.__professor

    @professor.setter
    def professor(self, professor):
        if professor is not None and not issubclass(type(professor), Professor):
            raise TypeError("parâmetro professor deve derivar da classe Professor")
        self.__professor = professor

#...
```

Boa prática. Para evitar redundâncias na verificação, vamos chamar os setters aqui.

issubclass

```python
from .conteudo_ministrado import ConteudoMinistrado
from .professor import Professor

class Disciplina:
    def __init__(self, nome, carga_horaria = 60, professor=None):
        self.__nome = nome
        self.carga_horaria = carga_horaria
        self.professor = professor
        self.__conteudos_ministrados = []

    @property
    def professor(self):
        return self.__professor

    @professor.setter
    def professor(self, professor):
        if professor is not None and not issubclass(type(professor), Professor):
            raise TypeError("parâmetro professor deve derivar da classe Professor")
        self.__professor = professor

#...
```

Caso o objeto sendo passado não seja um professor, ou algo que derive (seja um tipo de) professor, não vamos aceitar.

main.py

```python
from universidade.professor import Professor
from universidade.pessoa import Pessoa
from universidade.disciplina import Disciplina

try:
 p1 = Pessoa("Maria", 11111111111)
 t1 = Professor("João", 22222222222)
 disc1 = Disciplina("Orientação a Objetos", 60, p1)

 print(p1.nome)
 print(p1.cpf)

 print(t1.nome)
 print(t1.cpf)
 print(t1.salario)

 print(disc1.nome)
 print(disc1.professor.nome)
except ValueError as e:
 print("Erro de valor:", e)
except TypeError as e:
 print("Erro de tipo:", e)
except Exception as e:
 print("Erro genérico:", e)
```

main.py

Erro em tempo de execução. Somente professores e classes derivadas são aceitos.

```
from universidade.professor import Professor
from universidade.pessoa import Pessoa
from universidade.disciplina import Disciplina

try:
 p1 = Pessoa("Maria", 11111111111)
 t1 = Professor("João", 22222222222)
 disc1 = Disciplina("Orientação a Objetos", 60, p1)

 print(p1.nome)
 print(p1.cpf)

 print(t1.nome)
 print(t1.cpf)
 print(t1.salario)

 print(disc1.nome)
 print(disc1.professor.nome)
except ValueError as e:
 print("Erro de valor:", e)
except TypeError as e:
 print("Erro de tipo:", e)
except Exception as e:
 print("Erro genérico:", e)
```

Classes Abstratas

Tipos de Professor

Em uma universidade existem diversos tipo de professor.

- Professores Pesquisadores.
- Professores Substitutos.
- Professores Visitantes.
- ...

Cada um tem atributos específicos.

Não existem “Professores Genéricos”.

Classe Abstrata

```
from .pessoa import Pessoa
from abc import ABC, abstractmethod

class Professor(ABC, Pessoa):
  def __init__(self, nome, cpf, valor_hora = 60, carga_horaria = 40):
     super().__init__(nome, cpf)
     self.__valor_hora = valor_hora
     self.__carga_horaria = carga_horaria

  @abstractmethod
  def _calcular_bonus(self):
      pass
```

Classe Abstrata

Uma classe abstrata possui um ou mais **métodos abstratos**.

Precisa herdar de ABC (mais uma esquisitice do Python).

Um método abstrato é um **método incompleto**.

Temos sua assinatura (protótipo), mas não temos a implementação.

```
@abstractmethod
def _calcular_bonus(self):
    pass
```

Classe Abstrata

Uma classe abstrata possui um ou mais **métodos abstratos**.

Precisa herdar de ABC (mais uma esquisitice do Python).

Um método abstrato é um **método incompleto**.

Temos sua assinatura (protótipo), mas não temos a implementação.

Por definição, uma classe **abstrata é incompleta**.

Não devemos criar instâncias dessa classe.

Serve de esqueleto para criarmos outras classes através de herança.

```
@abstractmethod
def _calcular_bonus(self):
    pass
```

Classes Concretas

Classes que herdam de uma classe abstrata em implementam todos os métodos abstratos são classes Concretas.

Podemos instanciar objetos apenas de classes concretas.

ProfessorAjdunto

```python
from .professor import Professor

class ProfessorAdjunto(Professor):
  def __init__(self, nome, cpf, valor_hora=60, carga_horaria=40):
      super().__init__(nome, cpf, valor_hora, carga_horaria)
      self.__qtde_proj_pesquisa = 0

  @property
  def qtde_proj_pesquisa(self):
      return self.__qtde_proj_pesquisa

  @qtde_proj_pesquisa.setter
  def qtde_proj_pesquisa(self, qtde_proj_pesquisa):
      self.__qtde_proj_pesquisa = qtde_proj_pesquisa

  def _calcular_bonus(self):
      return self.__qtde_proj_pesquisa*1000
```

ProfessorSubstituto

```python
from .professor import Professor

class ProfessorSubstituto(Professor):
  def __init__(self, nome, cpf, valor_hora=60, carga_horaria=40):
      super().__init__(nome, cpf, valor_hora, carga_horaria)
      self.__anos_trabalho = 0

  @property
  def anos_trabalho(self):
      return self.__anos_trabalho

  @anos_trabalho.setter
  def anos_trabalho(self, anos_trabalho):
      self.__anos_trabalho = anos_trabalho

  def _calcular_bonus(self):
      return self.__anos_trabalho*300
```

main.py

```python
from universidade.professor_adjunto import ProfessorAdjunto
from universidade.professor_substituto import ProfessorSubstituto
from universidade.pessoa import Pessoa
from universidade.disciplina import Disciplina

try:
 ts = ProfessorSubstituto("João", 11111111111)
 ta = ProfessorAdjunto("Maria", 22222222222)
 disc1 = Disciplina("Orientação a Objetos", 60, ta)
 disc2 = Disciplina("Algoritmos", 60, ts)

 ts.anos_trabalho = 2
 ta.qtde_proj_pesquisa = 3

 print(disc1.nome)
 print(disc1.professor.nome)
 print(disc1.professor._calcular_bonus())

 print(disc2.nome)
 print(disc2.professor.nome)
 print(disc2.professor._calcular_bonus())

except ValueError as e:
 print("Erro de valor:", e)
except TypeError as e:
 print("Erro de tipo:", e)
except Exception as e:
 print("Erro genérico:", e)
```

main.py

```python
from universidade.professor_adjunto import ProfessorAdjunto
from universidade.professor_substituto import ProfessorSubstituto
from universidade.pessoa import Pessoa
from universidade.disciplina import Disciplina

try:
 ts = ProfessorSubstituto("João", 11111111111)
 ta = ProfessorAdjunto("Maria", 22222222222)
 disc1 = Disciplina("Orientação a Objetos", 60, ta)
 disc2 = Disciplina("Algoritmos", 60, ts)

 ts.anos_trabalho = 2
 ta.qtde_proj_pesquisa = 3

 print(disc1.nome)
 print(disc1.professor.nome)
 print(disc1.professor._calcular_bonus())

 print(disc2.nome)
 print(disc2.professor.nome)
 print(disc2.professor._calcular_bonus())

except ValueError as e:
 print("Erro de valor:", e)
except TypeError as e:
 print("Erro de tipo:", e)
except Exception as e:
 print("Erro genérico:", e)
```

Já aprendemos que não devemos fazer isso. Vamos fazer só para testar aqui. No próximo tópico vamos corrigir esse problema.

Faça você mesmo

A classe Professor herda de duas classes

Ao mesmo tempo.

Conceito de **herança múltipla**.

Pesquise sobre esse conceito, e implemente mais exemplos.

```
from .pessoa import Pessoa
from abc import ABC, abstractmethod

class Professor(ABC, Pessoa):
  def __init__(self, nome, cpf, valor_hora = 60, carga_horaria = 40):
     super().__init__(nome, cpf)
     self.__valor_hora = valor_hora
     self.__carga_horaria = carga_horaria
```

Inversão de Controle

Inversão de Controle

"Não nos chame. Nós chamaremos você."

Inverter o conceito comumente utilizado na programação procedural.

A função criada pelo programador chama funções prontas de bibliotecas.

Inversão de Controle

"Não nos chame. Nós chamaremos você."

Inverter o conceito comumente utilizado na programação procedural.

A função criada pelo programador chama funções prontas de bibliotecas.

Na inversão de Controle.

A biblioteca chama funções específicas do programador.

Inversão de Controle

Utilizado amplamente em frameworks (inclusive no Pytorch).

Pesquise na literatura.

Salário com Bônus

```python
from .pessoa import Pessoa
from abc import ABC, abstractmethod

class Professor(ABC, Pessoa):
  def __init__(self, nome, cpf, valor_hora = 60, carga_horaria = 40):
     super().__init__(nome, cpf)
     self.__valor_hora = valor_hora
     self.__carga_horaria = carga_horaria

  @abstractmethod
  def _calcular_bonus(self):
      pass

  @property
  def salario(self):
      #Carga semanal * custo * 4,5 semanas no mês + Bônus
      return int(self.__valor_hora * self.__carga_horaria * 4.5 + self._calcular_bonus())
```

Salário com Bônus

```python
from .pessoa import Pessoa
from abc import ABC, abstractmethod

class Professor(ABC, Pessoa):
  def __init__(self, nome, cpf, valor_hora = 60, carga_horaria = 40):
     super().__init__(nome, cpf)
     self.__valor_hora = valor_hora
     self.__carga_horaria = carga_horaria

  @abstractmethod
  def _calcular_bonus(self):
      pass

  @property
  def salario(self):
      #Carga semanal * custo * 4,5 semanas no mês + Bônus
      return int(self.__valor_hora * self.__carga_horaria * 4.5 + self._calcular_bonus())
```

Faça você mesmo

Coloque prints ou breakpoints nos métodos de cálculo de bônus.

Verifique que os métodos corretos são chamados automaticamente.

Sobrecarga de Operadores

Exemplo

```
from universidade.pessoa import Pessoa

i = 10
j = 10

p1 = Pessoa("João da Silva", 11111111111)
p2 = Pessoa("João da Silva", 11111111111)


if i == j:
  print(i,"é igual a", j)
else:
  print(i,"é diferente de", j)

if p1 == p2:
  print(p1.nome,"é igual a", p2.nome)
else:
  print(p1.nome,"é diferente de", p2.nome)
```

Exemplo

```python
from universidade.pessoa import Pessoa

i = 10
j = 10

p1 = Pessoa("João da Silva", 11111111111)
p2 = Pessoa("João da Silva", 11111111111)


if i == j:
  print(i,"é igual a", j)
else:
  print(i,"é diferente de", j)

if p1 == p2:
  print(p1.nome,"é igual a", p2.nome)
else:
  print(p1.nome,"é diferente de", p2.nome)
```

Console

```
10 é igual a 10
João da Silva é diferente de João da Silva
```

Comparadores

É possível sobrecarregar todos operadores básicos.

```
x<y através do método __lt__(self, y)
x<=y através do método __le__(self, y)
x==y através do método x.__eq__(self, y)
x!=y através do método x.__ne__(self, y)
x>y através do método x.__gt__(self, y)
x>=y através do método x.__ge__(self, y)
```

Exemplo

```python
from typing import cast

class Pessoa:

  def __eq__(self, other):
      return self._cpf == cast(Pessoa, other)._cpf

  def __ne__(self, other):
      return not self.__eq__(other)

  def __eq__(self, other):
      return self._cpf == cast(Pessoa, other)._cpf

  def __lt__(self, other):
      return self._cpf < cast(Pessoa, other)._cpf

  def __ge__(self, other):
      return not self.__lt__(other)

  def __gt__(self, other):
      return self._cpf > cast(Pessoa, other)._cpf

  def __le__(self, other):
      return not self.__gt__(other)
```

https://peps.python.org/pep-0484/#casts

Exemplo

```
from universidade.pessoa import Pessoa


lista = []
lista.append(Pessoa("João da Silva", 77777777777))
lista.append(Pessoa("Maria", 11111111111))
lista.append(Pessoa("Pedro", 55555555555))
lista.append(Pessoa("Camila", 88888888888))


# lista.sort()

for p in lista:
  print(p.nome)
```

Exemplo

```
from universidade.pessoa import Pessoa


lista = []
lista.append(Pessoa("João da Silva", 77777777777))
lista.append(Pessoa("Maria", 11111111111))
lista.append(Pessoa("Pedro", 55555555555))
lista.append(Pessoa("Camila", 88888888888))


# lista.sort()

for p in lista:
  print(p.nome)
```

Descomente essa linha e veja que a ordenação muda. Coloque prints nas funções de ordenação, e veja quais são chamadas pelo `sort()`.

Regras

Uma comparação de igualdade (e comparadores ‹, ›, !=, ...) devem seguir as seguintes regras.

Considerando três objetos, X, Y e Z, uma igualdade deve ser:

- Reflexiva: X == X deve ser verdadeiro.
- Simétrica: Se X == Y é verdadeiro, então Y == X é verdadeiro.
- Transitiva: Se X == Y é verdadeiro, e Y == Z é verdadeiro, então X ==Z é verdadeiro.
- Consistente: Múltiplas chamadas de X == Y deve retornar sempre o mesmo resultado se os objetos não foram alterados.

Leia mais em Bloch, (2018). Effective Java.

Sobrecarga de __str__

__str__

Existem vários outros métodos e operadores que podem ser sobrecarregados.

Pesquise!

Como um último exemplo, vamos sobrecarregar `__str__`.

__str__

O método `__str__` serve para retornar uma string que descreve o objeto.

Podemos sobrecarregar na classe Pessoa, para exibir suas informações de maneira formatada corretamente.

Exemplo

```
class Pessoa:
  def __str__(self) -> str:
      verificador = self._cpf%100
      atual = self._cpf//100
      terc = atual%1000
      atual = atual//1000
      seg = atual%1000
      prim = atual//1000

      return f"{self.nome} {prim:03d}.{seg:03d}.{terc:03d}-{verificador:02d}"
```

main.py

```python
from universidade.pessoa import Pessoa


lista = []
lista.append(Pessoa("João da Silva", 77777777777))
lista.append(Pessoa("Maria", 11111111111))
lista.append(Pessoa("Pedro", 55555555555))
lista.append(Pessoa("Camila", 88888888888))


lista.sort()

for p in lista:
  print(p)
```

Sobrecarregando na hierarquia

```
from .pessoa import Pessoa
from abc import ABC, abstractmethod

class Professor(ABC, Pessoa):
  def __init__(self, nome, cpf, valor_hora = 60, carga_horaria = 40):
     super().__init__(nome, cpf)
     self.__valor_hora = valor_hora
     self.__carga_horaria = carga_horaria


  def __str__(self) -> str:
      return f"{super().__str__()} R${self.salario},00"
```

Sobrecarregando na hierarquia

```
from .pessoa import Pessoa
from abc import ABC, abstractmethod

class Professor(ABC, Pessoa):
  def __init__(self, nome, cpf, valor_hora = 60, carga_horaria = 40):
     super().__init__(nome, cpf)
     self.__valor_hora = valor_hora
     self.__carga_horaria = carga_horaria


  def __str__(self) -> str:
      return f"{super().__str__()} R${self.salario},00"
```

Boa prática. Vamos chamar a função já pronta da classe pai (`super`), e só adicionar o que precisamos a mais.

main.py

```python
from universidade.pessoa import Pessoa
from universidade.professor_adjunto import ProfessorAdjunto

lista = []
lista.append(Pessoa("João da Silva", 77777777777))
lista.append(ProfessorAdjunto("Maria", 11111111111))
lista.append(ProfessorAdjunto("Pedro", 55555555555))
lista.append(Pessoa("Camila", 88888888888))


lista.sort()

for p in lista:
  print(p)
```

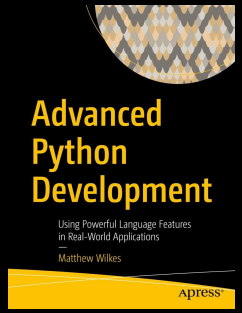
Exercícios

1. Reproduza os códigos e conceitos da aula em seu computador.
2. Tente criar novos experimentos para testar os conceitos aprendidos.

Referências

Python Docs: https://docs.python.org

Wilkes, Matthew. Advanced Python Development: Using Powerful Language Features in Real-World Applications. 2020.



Lott, Steven F.. Mastering Object-Oriented Python: Build Powerful Applications with Reusable Code Using OOP Design Patterns and Python 3.7. 2 Ed. 2019.